

CSCE 637 COMPLEXITY THEORY · MODULE 04 · LECTURE NOTES

NP-Completeness

One problem's difficulty, shared by thousands.

MODULE OUTCOMES

Define NP-hardness and NP-completeness and distinguish them.

Explain the Cook–Levin theorem and what it establishes.

Construct an NP-completeness proof with both halves.

Choose the right problem to reduce from.

Explain what NP-completeness does and does not predict.

1 The definitions, and the mechanism

NP-HARD IS ABOUT DIFFICULTY; NP-COMPLETE IS ABOUT DIFFICULTY AND MEMBERSHIP

L is NP-hard if every problem in NP reduces to it in polynomial time. L is NP-complete if it is NP-hard *and* is itself in NP.

So the halting problem is NP-hard and emphatically not NP-complete — it is undecidable, and therefore not in NP at all (CSCE 627 Module 05). **NP-hardness alone is a lower bound with no upper bound attached**, which is why the distinction is not pedantry.

Which is why a hardness proof is only half a proof. The membership half takes two sentences (Module 03 §1's certificate) **and is the half people omit** — and omitting it means you have proved a weaker and different statement.

And the consequence of completeness is the striking part: a polynomial-time algorithm for any single NP-complete problem yields one for every problem in NP, by composing with the reductions. **So thousands of problems from unrelated fields stand or fall together**, which is the fact that makes the classification worth having.

```
A ≤p B means: there is a polynomial-time computable f with
  x in A  ⇔  f(x) in B
```

```
TRANSITIVITY: if A ≤p B and B ≤p C then A ≤p C,
because the composition of two polynomials is a polynomial
(Module 02 section 3's closure property).
```

```
SO: once ONE problem is known to be NP-complete, proving a
second requires only a reduction FROM the first -- not from
all of NP.
```

```
every problem in NP  ≤p  SAT           (Cook-Levin)
SAT                  ≤p  3-SAT
3-SAT                ≤p  your problem
-----
every problem in NP  ≤p  your problem
```

```
WITHOUT TRANSITIVITY there would be no catalogue, and every
proof would have to start from the definition of NP. The
closure property of Module 02 is doing all the work here.
```

```
AND THE DIRECTION IS THE SAME AS CSCE 627 MODULE 06:
reduce the KNOWN HARD problem TO yours. Reducing YOURS to
3-SAT proves your problem is IN NP, which is the other half
and a different claim entirely.
```

Transitivity is why a catalogue exists — Garey and Johnson's three hundred problems are three hundred reductions chained through each other — **and the direction error here is exactly the same error as in CSCE 627 Module 06 §1, with the same fix: write down explicitly what reduces to what, every time.** **Note also that reducing your problem to 3-SAT is not wasted work:** it is how you solve it with a SAT solver (§4).

2 Cook–Levin, and the library

COOK–LEVIN: SAT IS NP-COMPLETE, BY ENCODING A COMPUTATION AS A FORMULA

Given any non-deterministic polynomial-time machine M and an input x , construct a propositional formula that is satisfiable exactly when M accepts x . That is the whole statement, and it is what bootstraps the field.

The formula's variables describe a computation *tableau* — a grid with one row per time step and one column per tape cell, recording the symbol in each cell, the head's position, and the machine's state.

Polynomially many rows and columns, so polynomially many variables.

And the clauses enforce local consistency: the first row encodes the start configuration with x on the tape; each row follows from the one above by one of M 's transitions, checked in 2×3 windows; and some row is accepting. **Every constraint is local**, which is why it fits in a formula of polynomial size.

So satisfiability of that formula is the existence of an accepting computation of M on x . The theorem works because propositional logic is expressive enough to describe computation step by step — which is CSCE 627 Module 10 §2's arithmetisation idea in a different setting, and is also exactly what bounded model checking exploits (CSCE 627 Module 08 §3).

Reduce from	When your problem involves
3-SAT	Logical constraints, or nothing else fits. The default, and the usual answer.
Vertex cover, independent set, clique	Selecting a set of objects subject to pairwise conflicts — and the three are trivially irreducible, so pick whichever matches your framing.
Hamiltonian cycle or path	Ordering or sequencing, visiting everything exactly once — routing, scheduling without repetition, genome assembly.
3-dimensional matching	Grouping into triples — and it is the right source for many partition-like problems where 3-SAT is awkward.
Subset sum, partition	Numbers and arithmetic targets — and note these are the pseudo-polynomial ones (Module 01 §2), so the hardness depends on large numbers.
Graph colouring	Assignment under conflict constraints — register allocation (CSCE 605), frequency assignment, timetabling.
Set cover	Covering a universe with chosen subsets — and note it is also the canonical hardness-of-approximation case (Module 11 §3).

Choose the source problem structurally closest to your own — the reduction will be dramatically shorter and far less error-prone — **which is CSCE 627 Module 06 §3's library argument, with a much bigger library.** Garey and Johnson's appendix is the library, and an hour spent finding the right source saves a day constructing a reduction from the wrong one.

3 Constructing a proof

1. MEMBERSHIP. Name the certificate and the check. Two sentences (Module 03 section 1). DO THIS FIRST, because if it fails then your problem is not in NP and you are proving something other than what you intended.
2. CHOOSE the source problem from section 2's table.
3. CONSTRUCT f : an instance of the source $\rightarrow$ an instance of your problem.
4. PROVE BOTH DIRECTIONS:
 source YES $\Rightarrow$ your problem YES
 your YES $\Rightarrow$ source YES
 The SECOND direction is where proofs fail. It requires showing that EVERY solution to your constructed instance arises from a solution to the source -- including solutions your construction did not anticipate and does not want.
5. VERIFY f runs in polynomial time. Usually obvious, and state it anyway: a reduction that blows up the instance size exponentially is not a reduction.

THE COMMON ERRORS

omitting step 1 $\rightarrow$ you proved NP-hardness only
reversing step 3 $\rightarrow$ you proved membership, not hardness
hand-waving step 4's second direction $\rightarrow$ the proof is simply incomplete, and this is the usual failure

Step 4's second direction is where real proofs fail. Unintended solutions to your constructed instance are the thing to rule out — you built a graph intending the solver to find a particular kind of structure, and you must show no other structure satisfies the constraints, which frequently requires adding gadgets purely to forbid alternatives. **A proof that handles only the forward direction is not a flawed proof; it is half a proof.**

4 What NP-completeness predicts

NP-COMPLETENESS IS A STARTING POINT, NOT A VERDICT

It says: no polynomial-time algorithm is known, and finding one would resolve a famous open problem in the affirmative. That is strong evidence of difficulty and it is not a proof of intractability — which is Module 03 §2's conditionality, and it is the honest reading.

It says nothing whatsoever about your particular instances. SAT solvers handle instances with millions of variables (CSCE 625 Module 08 §2); **mixed-integer programming solvers improved by a factor of a million in thirty years** (CSCE 669 Module 09 §4). **Both on NP-complete problems.**

It does not tell you the approximation picture, which may be excellent (knapsack has a fully polynomial approximation scheme) or hopeless (clique admits no constant factor) — **and those two are both NP-complete, so the label does not distinguish them. That is Module 11.**

Nor whether a parameter makes it easy. Vertex cover is NP-complete and fixed-parameter tractable, solvable in time exponential only in the solution size — which is Module 12, and is the most practically useful material in the course. **So the label begins the engineering rather than concluding it.**

- **Try a solver. SAT, SMT, mixed-integer, or constraint programming** (CSCE 669 Module 13 §1) — **and your instances may simply not be hard**, which you will not know until you try.
- **Look for an approximation guarantee** (CSCE 669 Module 10, and Module 11 here) **and for the matching hardness-of-approximation bound**, so you know whether a better ratio is available.
- **Look for a parameter** (Module 12) — treewidth, solution size, number of distinct values, a structural restriction. **This is the most reliably productive step.**
- **Check whether your special case is in P.** Planar, bipartite, bounded-degree, interval, chordal, series-parallel — **frequently it is**, and the restriction is frequently one your instances satisfy anyway.
- **And compute a bound even if you ultimately use a heuristic** (CSCE 669 Module 13 §3) — **'within 7% of optimal' is a categorically stronger claim than a number. All five are routinely productive**, which is why **'NP-complete, therefore hopeless' is the wrong inference and the common one.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapter 2, and Sipser chapter 7 (free draft)	Cook–Levin with the tableau construction , and the standard reductions.
Garey & Johnson — Computers and Intractability	§2's library, in full — three hundred problems with their status, and the guide to constructing reductions in chapter 3. Library copy, and still the reference.
Karp — Reducibility Among Combinatorial Problems (free)	The 1972 paper that produced the first twenty-one , and turned Cook's single theorem into a field.

Resource	What it covers for this module
Levin — Universal sequential search problems (translation, free)	The independent 1973 formulation , which is two pages and frames the question differently.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State the difference between NP-hard and NP-complete** and give a problem that is the first and not the second.
- 2 **Prove that polynomial reductions compose**, with the exponent arithmetic.
- 3 **Work through the Cook–Levin tableau construction** for a two-state machine and a three-cell tape.
- 4 **Reduce 3-SAT to vertex cover** yourself, both directions.
- 5 **Reduce 3-SAT to a problem from your own work**, following §3's five steps.
- 6 **Write the membership half explicitly**, with the certificate.
- 7 **Find the unintended solutions** to your constructed instance and add a gadget to forbid them.
- 8 **Encode an NP-complete problem for a SAT solver** and run it on instances of growing size. Report where it breaks.
- 9 **Find a special case of your problem that is in P** and prove it.
- 10 **Apply all five of §4's steps** to one problem and report which helped.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish NP-hard from NP-complete, with an example.
- 2 Why is a hardness proof only half a proof?
- 3 Why does transitivity matter, and what would be lost without it?
- 4 Explain the Cook–Levin construction.
- 5 Why is logic expressive enough for it to work?
- 6 Name seven problems to reduce from and when to use each.
- 7 Give the five steps of a completeness proof and the three common errors.
- 8 Where do real proofs fail, and why?
- 9 Name four things NP-completeness does not tell you.
- 10 Give five things to do after proving it.

